

Laboratorio de Electrónica III / 25.24

TPL 4

Ramiro Nieto Abascal
rnietoabascal@itba.edu.ar

3 de mayo de 2026



ÍNDICE

1. Introducción	3
1.1. Resumen	3
1.2. Presentación de los resultados	3
2. Ejercicio 1: Filtro anti-alias	4
2.1. Diseño de filtro	4
2.2. Implementación del sistema & verificación empírica del funcionamiento	6
2.3. Tiempo de procesamiento	8
3. Ejercicio 2: filtrado por convolución	9
3.1. Overlap and save	10
3.2. Implementación	12
4. Ejercicio 3: Analizador de THD	14
4.1. Diseño de bloques	14
4.1.1. Filtro comb	14
4.1.2. Pasa-bajos Recuperación de primeros 10 armónicos	15
4.1.3. Filtro pasabajos Recuperación de fundamental	17
4.1.4. Simulación	18
Referencias	21

INTRODUCCIÓN

1.1 RESUMEN

El presente informe corresponde al Trabajo Práctico N°1: “Return Ratio”, de la materia Electrónica III (25.21). En esta práctica se realiza un breve comentario inicial sobre el modelo asintótico que será utilizado para describir el comportamiento de los sistemas estudiados, junto con una descripción del procedimiento necesario para obtener los parámetros dispuestos allí, al igual que un potencial ensayo para determinarlos experimentalmente.

1.2 PRESENTACIÓN DE LOS RESULTADOS

En las siguientes secciones se presentan los resultados obtenidos durante el desarrollo de la práctica. En primera instancia, de corresponder, se hará mención del comportamiento esperado en función de las bases teóricas planteadas en clase. Luego, se hará una descripción de las mediciones realizadas y las conclusiones que se pueden extraer de estas. Finalmente, de corresponder, se detallarán propuestas de vías de investigación a futuro con tal de obtener un mejor entendimiento del contenido tratado.

EJERCICIO 1: FILTRO ANTI-ALIAS

Se desea realizar un down-sampling de $f_s = 10kHz$ a $f'_s = 2kHz$. La señal de interés está acotada en frecuencia hasta $970Hz$; de este modo, con tal de evitar aliasing, se ha de diseñar un filtro pasa-bajos.

En vista al problema planteado, se solicita que la atenuación en $970Hz$ resulte de $6dB$ y una atenuación de al menos $50dB$ a partir de $f'_s/2 = 1000Hz$.

2.1 DISEÑO DE FILTRO

En función de los requerimientos previamente establecidos se diseña un filtro FIR mediante el método de la serie de Fourier. En este método resulta relevante la elección de la ventana a emplear en vista de optimizar el orden del filtro, donde varía, entre otras características, la atenuación del primer lóbulo lateral y la pendiente del lóbulo principal; afectando así el ancho de la banda de transición y la ganancia en la banda de atenuación. En vista a esta elección y en función de los requerimientos del filtro a implementar, se realizó una búsqueda entre las siguientes ventanas:

- Rectangular.
- Bartlett.
- Hann.
- Hamming.
- Blackman.

En función de este requerimiento, se implementó una búsqueda del número de taps necesario para cada una de ellas, con un máximo de 5000 (el proceso corresponde a buscar la función tal que la atenuación a partir de $f'_s/2 = 1000Hz$ sea de $50dB$). Obteniendo así:

Tabla 2.1: Comparación de ventanas para el diseño del filtro FIR anti-alias

Ventana	Orden mínimo (taps)	Atenuación mínima [dB]	Cumple
Rectangular (boxcar)	–	–	No
Bartlett	–	–	No
Hann	761	50.10	Sí
Hamming	527	50.22	Sí
Blackman	741	50.21	Sí

donde además, se forzó en la búsqueda un número impar de taps tal que el filtro obtenido sea de fase lineal (por ende, group delay constante).

En función de la tabla 2.1 se decide emplear la ventana de Hamming en el diseño del filtro. Bajo esta configuración se obtiene la siguiente respuesta en frecuencia:

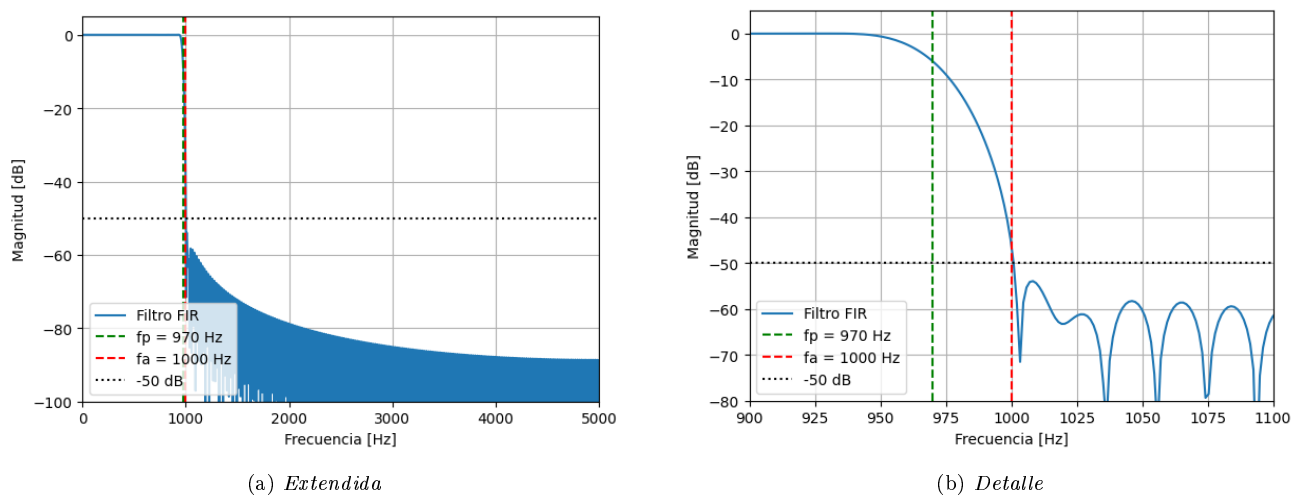


Figura 2.1: *Respuesta en frecuencia / Módulo*

Donde se verifica el comportamiento esperado (no se muestra un detalle en las inmediaciones de 970Hz dado que la forma en la que se implemento garantiza la atenuación de 6dB allí).

En tanto a otras características fundamentales del filtro se puede analizar la fase:

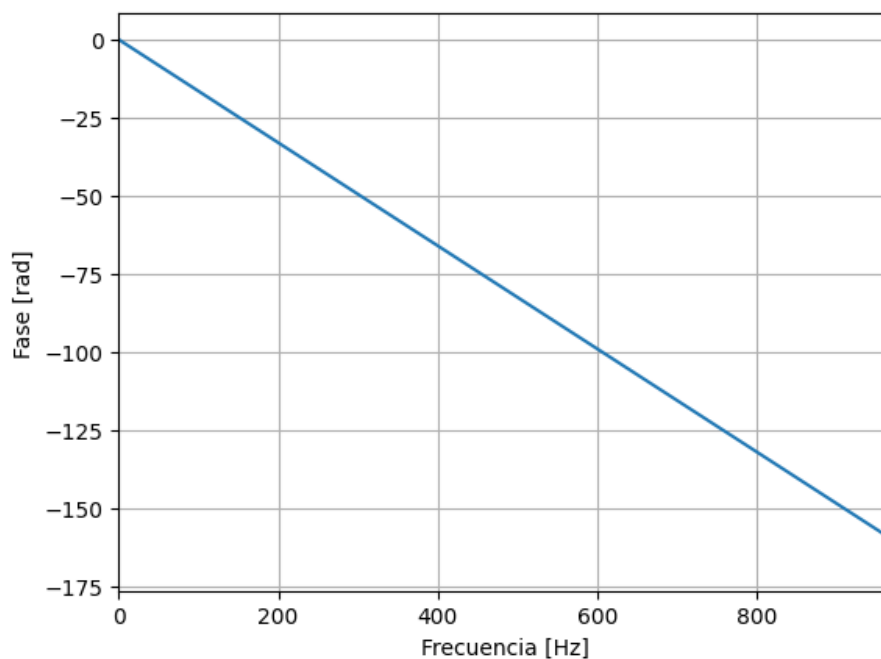


Figura 2.2: *Respuesta en frecuencia / Fase*

siendo relevante observar que efectivamente se dispone de fase lineal. Finalmente también resulta de interés observar la respuesta impulsiva del sistema:

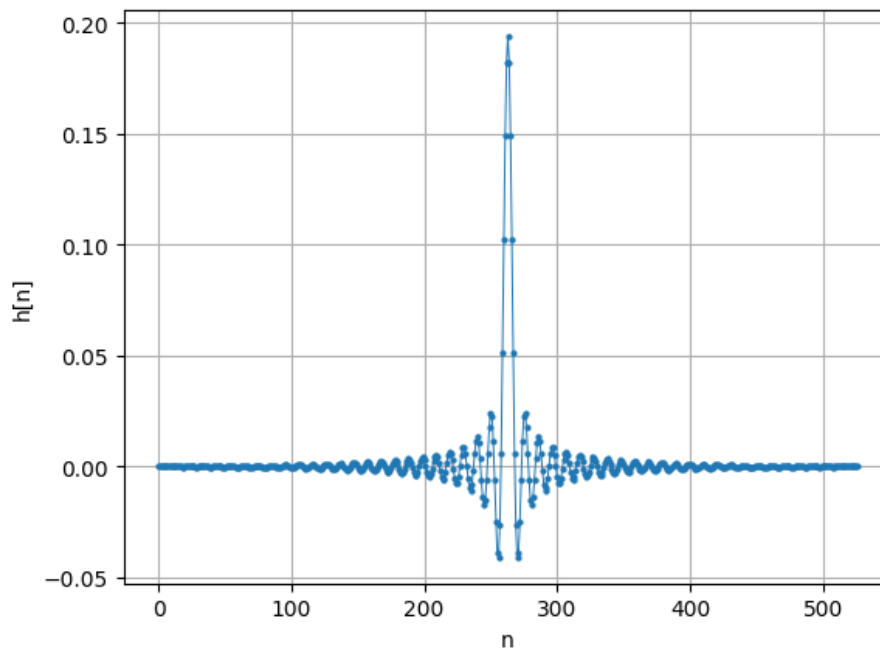


Figura 2.3: *Respuesta impulsiva.*

donde se observa la simetría para respecto a $N/2$.

2.2 IMPLEMENTACIÓN DEL SISTEMA & VERIFICACIÓN EMPÍRICA DEL FUNCIONAMIENTO

Se realizó la implementación del filtro:

```

1 def fir_filter(x, h):
2     x = np.asarray(x)
3     h = np.asarray(h)
4
5     N = len(h)
6     L = len(x)
7
8     y = np.zeros(L)
9
10    for n in range(L):
11        for k in range(N):
12            if n-k >= 0:
13                y[n] += h[k] * x[n-k]
14
15    return y

```

Listing 1: *Implementación del filtro.*

tras la cual puede realizarse el down-sampling deseado:

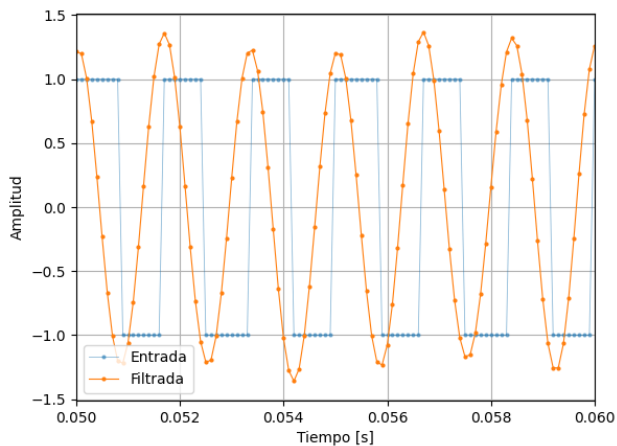
```

1 # --- filtrado ---
2 y = fir_filter(x, h)
3
4 #--- down-sampling ---
5 N_ds = 5
6
7 y_ds = y[::N_ds]
8 t_ds = t[::N_ds]

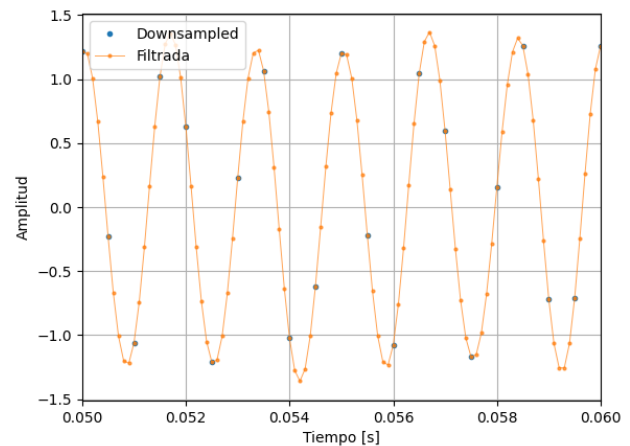
```

Listing 2: *Filtrado + down-sampling.*

En el marco temporal, estas operaciones corresponden con las siguientes señales:



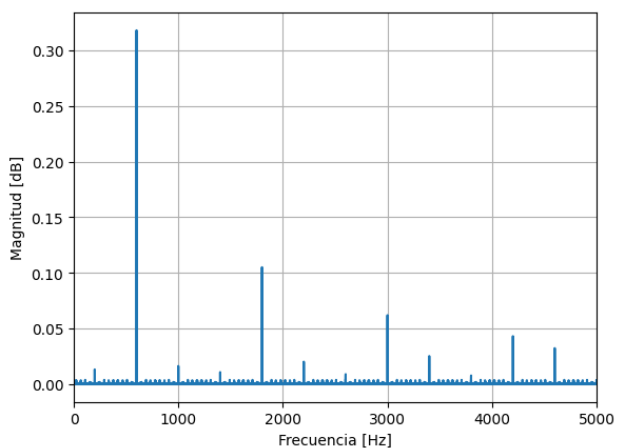
(a) *Entrada + señal filtrada.*



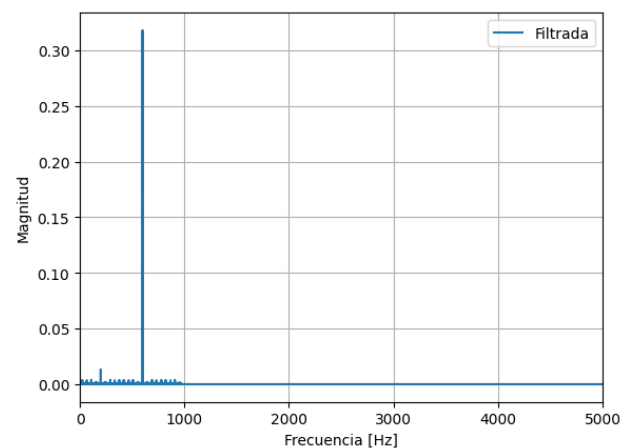
(b) *Señal filtrada + downsampling.*

Figura 2.4: Comparativa de señales en marco temporal

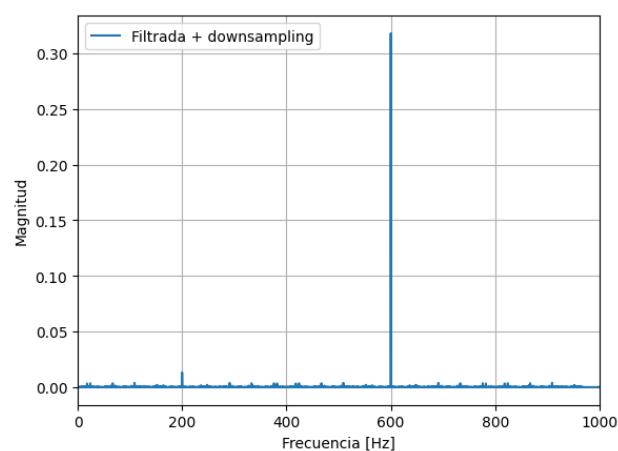
Esto se puede analizar desde el marco espectral realizando una FFT sobre las señales previamente indicadas:



(a) *Entrada*



(b) *Filtrada*



(c) *Filtrada + down-sampling*

Figura 2.5: Comparativa de señales en el dominio espectral

donde resulta relevante destacar que, por un lado, el cambio del rango en el eje corresponde a la modificación de la frecuencia de muestreo. Por otro lado, se observan armónicos que no eran esperados en función de la entrada indicada, estos se deben a que no se filtró la señal cuadrada a la hora de muestrearse, es decir, la entrada al sistema dispone de aliasing.

2.3 TIEMPO DE PROCESAMIENTO

Se solicita calcular el tiempo de procesamiento del sistema. Para ello se realizaron 5 ejecuciones del código previamente indicado obteniendo:

Tabla 2.2: *Tiempos de procesamiento del filtro FIR*

#(Ejecución)	Tiempo [s]	Tiempo por muestra [μ s]
1	19.135894	191.36
2	18.456400	184.56
3	18.152865	181.53
4	18.438790	184.39
5	18.219928	182.20
Promedio	18.480776	184.808

EJERCICIO 2: FILTRADO POR CONVOLUCIÓN

Se desea implementar el filtro desarrollado en la sección anterior mediante convolución. Este algoritmo tiene base en realizar una FFT de la entrada y respuesta impulsiva, para luego realizar el producto de las mismas en el marco espectral y anti-transformar (IFFT) el resultado, obteniendo así la salida del filtro:

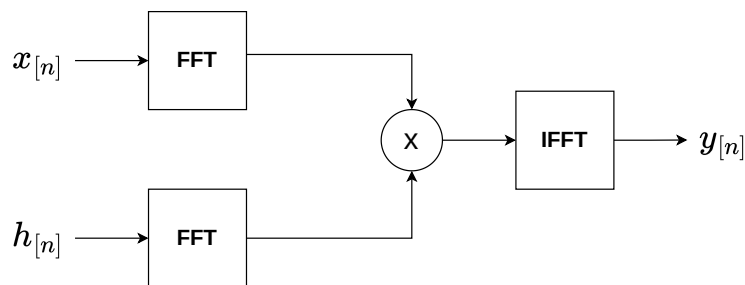


Figura 3.1: Esquema básico de filtrado por convolución

En función de que el algoritmo de FFT es $O(n \cdot \log(n))$ se busca optimizar el cálculo.

Si bien la operatoria parecería estar completamente definida bajo este esquema, si se requiere procesar la entrada en bloques se han de tener ciertas consideraciones. Para llegar a ellas debemos partir de la cuenta que se ha de realizar en el marco temporal. Si el filtro dispone de M coeficientes:

$$y[n] = \sum_{k=0}^{M-1} h[k] \cdot x[n - k]$$

lo cual corresponde con la operatoria habitual. Sin embargo, a la hora de emplear la DFT (o, en su defecto, la FFT) se realiza una extensión periódica de la señal, por lo que la operación desplazamiento no se condice con la expresión previamente indicada. A la operación que se debe realizar se la conoce como *convolución circular*, mientras que la que se realiza habitualmente lleva el nombre de *convolución lineal*. Esto parecería no tener relevancia para con la cuenta que se ha de efectuar en esta técnica, pues le compete al marco temporal y no al espectral (donde en principio tiene pie este proceso); sin embargo, esto es relevante para determinar la longitud de la FFT a realizar.

De querer filtrar una señal $x[n]$ de longitud L mediante el filtro previamente descrito ($h[n]$ de longitud M), la salida asociada tendría una longitud $L + M - 1$. De este modo, si se toma una cantidad de muestras N para realizar la FFT tal que sea menor que $L + M - 1$ el cálculo sería erróneo. Para visualizarlo tomemos un ejemplo sencillo:

$$x[n] = 1, 1$$

$$h[n] = 1, 1$$

aquí $L=M=2$. Si realizamos la convolución lineal y calculamos $y[0]$, observamos que:

$$y[0] = x[0] \cdot h[0] + x[1] \cdot h[-1] = 1 \cdot 1 + 1 \cdot 0 = 1$$

si ahora tomamos $N = 2$, la relación que se mantiene es la misma, pero la señal temporal asociada a la señal espectral obtenida con la DFT no tiene $h[-1] = 0$, sino que $h[1] = 1$. De este modo:

$$y[0] = x[0] \cdot h[0] + x[1] \cdot h[-1] = 1 \cdot 1 + 1 \cdot 1 = 2 \neq 1$$

de este modo, resulta evidente el problema que se estaba desarrollando. En vista a esto, la conclusión es que se ha de realizar un padding con ceros a la hora de calcular la FFT con tal de que la cuenta en el marco espectral se condiga con la cuenta deseada en el marco temporal. Consecuentemente se realiza una FFT de al menos $L + M - 1$ muestras. Siendo que la FFT es un algoritmo de cálculo de la DFT que resulta eficiente en con tamaños potencias de dos, es posible ajustar N tal que se den ambas condiciones simultáneamente. En este caso particular, siendo $L = 1024$ y $M = 527$, se puede tomar $N = 2048$.

Hasta aquí se concluyo la necesidad de añadir ceros adicionales para poder efectuar la cuenta correspondiente sin error. No obstante resta analizar dónde se han de colocar estos ceros, al igual que como se han de procesar los subsecuentes bloques. Existen dos técnicas que definen estos comportamientos:

- Overlap and save.
- Overlap and add.

En este informe se desarrollará el primero de ellos.

3.1 OVERLAP AND SAVE

Por un lado, la respuesta impulsiva a transformar dispone de $N-M$ ceros adicionales en su cola:

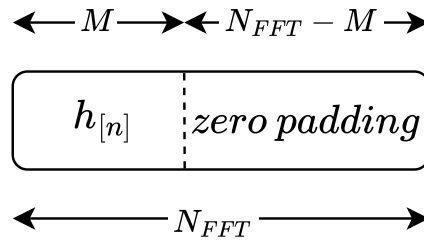


Figura 3.2: Respuesta impulsiva a transformar.

notar que la transformación de este bloque se efectúa una única vez (si se dispone de HW dedicado se puede almacenar únicamente esta transformada en lugar de los coeficientes de la respuesta impulsiva).

En tanto a la señal de entrada, se han de tomar también N muestras, siendo relevante estudiar cuales de ellas. En la primera iteración se colocan $M - 1$ ceros al comienzo del bloque, de lo contrario la cuenta efectiva con la FFT emplearía los elementos de la cola del bloque (recordar la extensión periódica que realiza esta operación). Luego de este padding se dispone de las primeras L muestras a procesar y se completa el bloque con ceros hasta alcanzar N muestras. En las siguientes iteraciones, al igual que cuando se realiza la convolución lineal, se han de tomar las $M-1$ muestras del bloque previo, junto con las siguientes L muestras a procesar + zero padding:

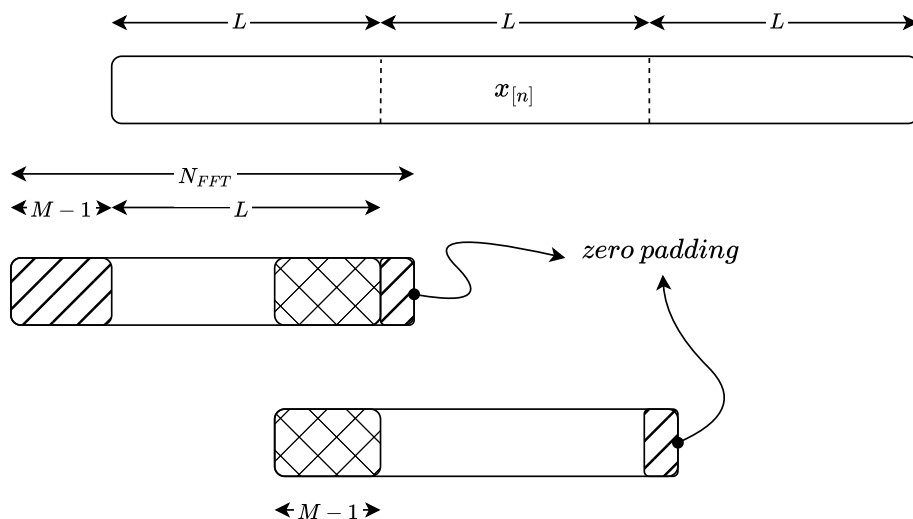


Figura 3.3: Armado de bloques de entrada.

Finalmente para con la salida, en cada iteración se obtiene N muestras a partir de la IFFT, de estas N muestras las primeras $M - 1$ no son válidas (por la misma razón que se comentó anteriormente), al igual que las últimas $N - (L - M - 1)$:

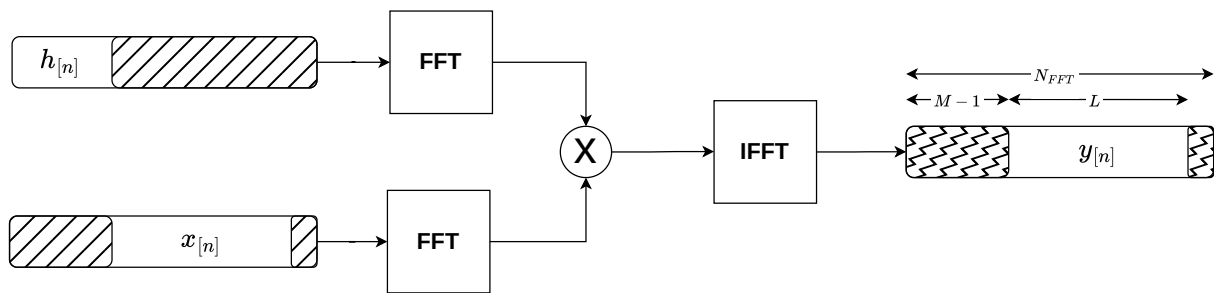


Figura 3.4: Diagrama de flujo.

3.2 IMPLEMENTACIÓN

Con tal de implementar el método desarrollado se hizo uso del siguiente código:

```

1  import numpy as np
2  import time
3
4  # --- Datos de entrada (asumiendo que ya existen) ---
5  # h: tus 527 coeficientes
6  # x: tu señal de entrada
7  # y_ref: el resultado de ejercicio 1 (convolucion lineal) para comparar
8
9  L = 1024
10 M = len(h)
11 N = 2048 # Potencia de 2 recomendada para h=527 y L=1024
12
13 # 1. Pre-cálculo: FFT del filtro (solo una vez)
14 H = np.fft.fft(h, n=N)
15
16 # Inicializacion
17 salida_final = []
18 tiempos_bloque = []
19 pasado = np.zeros(M - 1) # M-1 ceros iniciales
20
21 # 2. Procesamiento por bloques
22 for i in range(0, len(x), L):
23     start_time = time.perf_counter()
24
25     # Tomamos el bloque actual de la señal
26     bloque_actual = x[i : i + L]
27
28     # Si el ultimo bloque es mas chico que L, lo rellenamos con ceros
29     if len(bloque_actual) < L:
30         bloque_actual = np.pad(bloque_actual, (0, L - len(bloque_actual)))
31
32     # Armado del bloque de entrada N=2048: [pasado(526) + actual(1024) +
33         # ceros(498)]
34     # Usamos np.concatenate o simplemente pasamos el tamaño a np.fft.fft
35     entrada_fft = np.concatenate([pasado, bloque_actual])
36
37     # Procesamiento espectral
38     X = np.fft.fft(entrada_fft, n=N)
39     Y = X * H
40     y_bloque = np.fft.ifft(Y).real # Nos quedamos con la parte real
41
42     # --- OVERLAP AND SAVE: Salvar desde el indice M-1 hasta (M-1 + L) ---
43     # Esto extrae exactamente las 1024 muestras limpias
44     bloque_limpio = y_bloque[M-1 : M-1+L]
45     salida_final.extend(bloque_limpio)
46
47     # Actualizamos el pasado para la siguiente iteracion
48     pasado = bloque_actual[-(M-1):]
49
50     end_time = time.perf_counter()
51     tiempos_bloque.append(end_time - start_time)

```

```
51 # 3. Mediciones y Comparacion
52 salida_final = np.array(salida_final[:len(x)]) # Recortamos al largo original
53     si es necesario
54 tiempo_total = sum(tiempos_bloque)
55 tiempo_por_muestra = tiempo_total / len(x)
56
57 print(f"Tiempo total: {tiempo_total:.4f} s")
58 print(f"Tiempo por muestra: {tiempo_por_muestra:.8f} s")
59
60 # Comparacion con el ejercicio 1 (y_ref)
61 # Nota: y_ref debe ser recortado o comparado hasta len(x)
62 error = np.linalg.norm(salida_final - y_ref[:len(x)])
63 print(f"Error acumulado contra convolucion: {error}")
```

Listing 3: Código empleado en la implementación.

de este modo, se obtuvo un tiempo total de 38ms, equivalente a $0,38\mu s/muestra$ (con un error acumulado de 2×10^{-13} que se condice con un error computacional). Esta es una gran mejoría con respecto a los tiempos obtenidos en el ejercicio 1 (18.48s de procesamiento netos + $184,8\mu s$). Esta gran diferencia se debe a, por un lado, la diferencia en la complejidad computacional ($O(n^2)$ vs $O(n \log(n))$), junto con un mejor empleo de los recursos (probablemente las funciones empleadas para el cálculo de la FFT e IFFT estén optimizadas, no así el código implementado en el ejercicio 1 con los dos for anidados).

EJERCICIO 3: ANALIZADOR DE THD

Se requiere diseñar un dispositivo de submetering que calcule en tiempo real el THD de la tensión de línea de una subestación eléctrica.

Para ello se ha de implementar el siguiente esquema:

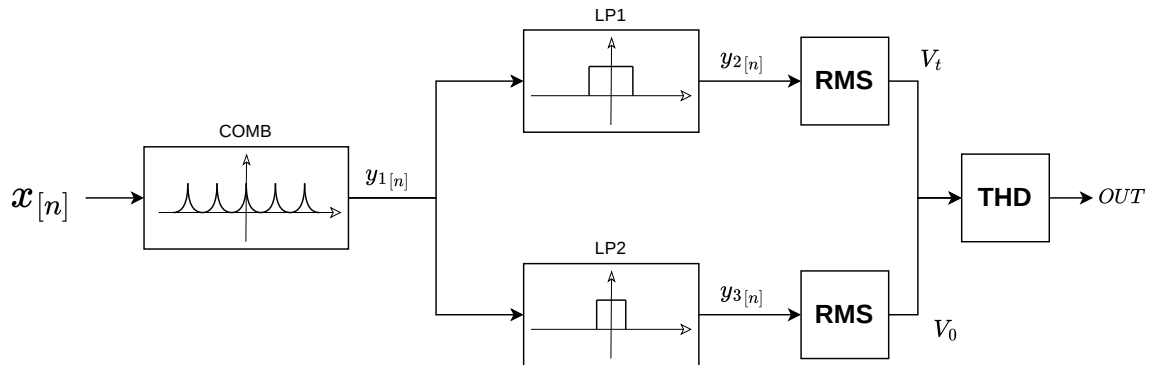


Figura 4.1: Esquema general a implementar.

4.1 DISEÑO DE BLOQUES

En este apartado se indicaran las características de cada uno de los bloques enseñados en la figura 4.1 y se realizara el diseño de los mismos.

4.1.1 FILTRO COMB

Dado que se desea analizar los armónicos de la red, debemos de diseñar un filtro que disponga de ganancia unitaria para estos y las componentes restantes. En función de ello se solicita la característica de atenuar al menos 10 veces las frecuencias múltiplo impar de 25Hz.

En vista de diseñar tal filtro, se dispone de la siguiente ecuación en diferencias:

$$y_1[n] = \alpha x[n] + \beta y_1[n - k] \quad (1)$$

donde hemos de determinar α , β y k . Para ello podemos realizar un análisis desde la función transferencia del sistema:

$$H(z) = \frac{Y_1(z)}{X(z)} = \frac{\alpha}{1 - \beta z^{-k}} = \frac{\alpha \cdot z^k}{z^k - \beta} \quad (2)$$

donde resulta evidente que se dispone de k ceros en el origen y k polos repartidos uniformemente en una circunferencia de radio $\beta^{1/k}$. De este modo, k ha de corresponder con la cantidad de armónicos de 50Hz que entran en $f_s = 10kHz$; de este modo, $k=200$. Luego debemos de ajustar

α y β tal de obtener las ganancias deseadas. En función de ello se puede analizar la respuesta en frecuencia del sistema:

$$|H(\omega)| = \frac{\alpha}{\sqrt{(\cos(\omega k) - \beta)^2 + \sin^2(\omega k)}} = \frac{\alpha}{\sqrt{1 + \beta^2 - 2 \cdot \beta \cdot \cos(\omega k)}} =$$

de este modo, la ganancia en los picos esta dada cuando $\omega = 2\pi/k$, tal que:

$$\frac{\alpha}{1 - \beta} = 1 \implies \alpha = 1 - \beta \quad (3)$$

mientras que los múltiplos impares de 25Hz corresponden a $\omega = \frac{2\pi \cdot m \cdot 25Hz}{2\pi \cdot f_s} = m \cdot \frac{\pi}{k}$ con m impar. Por ende:

$$\frac{\alpha}{1 + \beta} = 0,1 \implies \alpha = 0,1 + 0,1\beta \quad (4)$$

resolviendo el sistema de ecuaciones se obtiene $\alpha = 2/11$ y $\beta = 9/11$.

En función de los parámetros obtenidos puede graficarse la respuesta en frecuencia asociada:

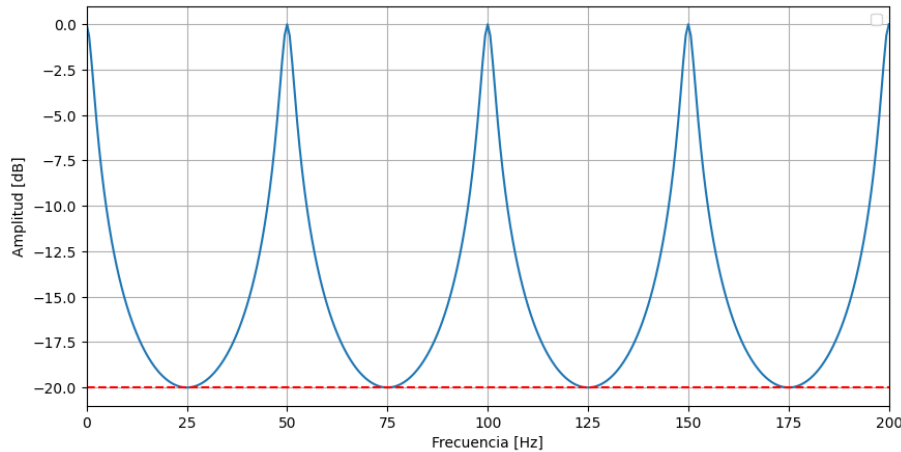


Figura 4.2: Módulo de respuesta en frecuencia de filtro comb.

4.1.2 PASA-BAJOS | RECUPERACIÓN DE PRIMEROS 10 ARMÓNICOS

En segundo lugar se debe diseñar un filtro pasa-bajos tal que se recuperen los primeros 10 armónicos, donde la atenuación subsecuente ha de ser de al menos 20dB. Para lograr este acometido se ha de diseñar un filtro FIR mediante el método de la serie de Fourier (análogo al ejercicio 1). En función de ello se ha de decidir la ventana óptima para la tarea.

En este caso particular la decisión es más compleja que las circunstancias planteadas para el ejercicio 1. Por un lado se ha de considerar que la ganancia en la banda de atenuación no es tan restrictiva como lo fue en el primer caso. Una atenuación de 20dB es lo suficientemente pequeña tal que se pueda emplear una ventana rectangular, por lo que potencialmente se puede aprovechar

la gran caída de la misma para optimizar el número de taps que ha de disponer el filtro. Sin embargo, esta ventana dispone de un problema fundamental: el fenómeno de Gibbs. La respuesta en frecuencia presenta oscilaciones en la banda pasante. Estas oscilaciones potencialmente pueden llegar a ser críticas para el cálculo del THD, dado que en el esquema de la Figura 4.1 no se dispone de un método para discernir de los armónicos y compensar esta atenuación. En función de ello se puede decidir optar por otra ventana que no disponga de esta problemática, como puede ser una ventana de Hamming. Esta ventana no dispone de estas oscilaciones pronunciadas, sin embargo el número de taps a emplear potencialmente crece drásticamente respecto de la primera opción. En función de ello se sugiere el uso de una tercera opción que no fue analizada en el primer ejercicio: la ventana de Kaiser. Esta ventana dispone de un parámetro adicional (β), que le confiere un grado de libertad que permite ajustar la caída y atenuación del lóbulo lateral. De este modo podemos intentar encontrar un punto intermedio entre los beneficios que confieren las ventanas previamente mencionadas, optimizando para nuestro beneficio.

En función de ello se realizó un código en Python tal que se trimmeo el número de taps y β (en el caso de la ventana de Kaiser), con tal de cumplir las especificaciones requeridas. Para la frecuencia de cruce que requiere como parámetro de entrada la función `signal.firwin()` se empleó $f_c=525\text{Hz}$ (notar que si se usa 500Hz potencialmente se estaría incurriendo en un error al atenuar un armónico relevante para el cálculo del THD). De este modo se obtuvieron:

Tabla 4.1: *Parámetros de diseño finales para el filtro pasabajos $y_2[n]$ ($f_c = 530\text{Hz}$)*

Ventana	Orden (N)	Parámetro Extra	Retardo de Grupo ($N/2$)
Rectangular	221	-	110.5 muestras
Hamming	451	-	225.5 muestras
Kaiser	281	$\beta = 1,8$	140.5 muestras

que se condice con las siguientes respuestas en frecuencia:

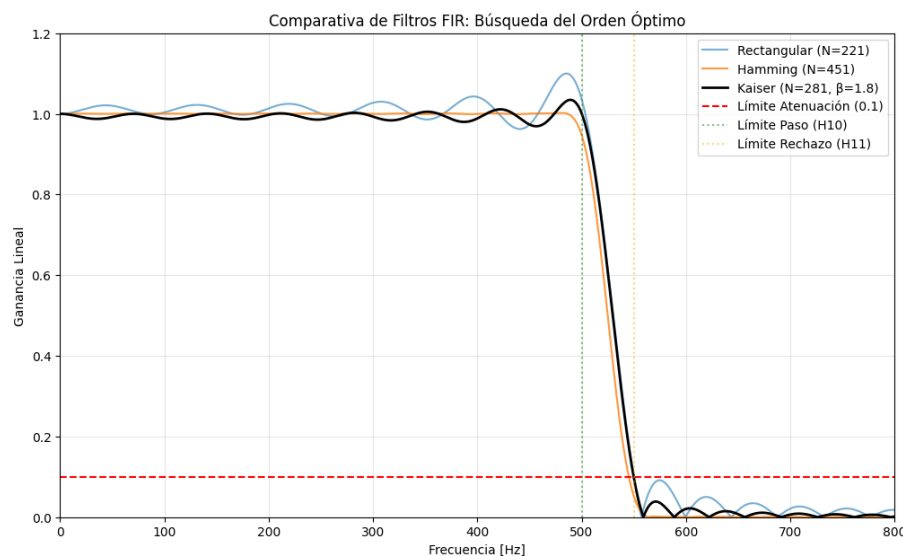


Figura 4.3: *Módulo de respuesta en frecuencia de filtro pasa-bajos.*

En este contexto es relevante las ganancias en las frecuencias de interés:

Tabla 4.2: Comparativa de ganancia lineal en los armónicos según el tipo de ventana ($f_s = 10\text{kHz}$)

Armónico	Frecuencia [Hz]	Rectangular	Hamming	Kaiser
H1	50	1.0201	1.0001	0.9921
H2	100	1.0036	1.0005	0.9880
H3	150	1.0131	1.0008	0.9985
H4	200	1.0136	1.0007	0.9976
H5	250	1.0008	1.0004	0.9860
H6	300	1.0274	0.9998	0.9931
H7	350	0.9862	0.9995	1.0049
H8	400	1.0420	0.9998	0.9885
H9	450	0.9733	1.0004	0.9735
H10	500	1.0366	0.9478	1.0013
H11	550	0.0998	0.0524	0.0986
H12	600	0.0020	0.0002	0.0205

de donde es posible extraer el error que se está cometiendo debido a las oscilaciones y/o la ganancia en el último armónico:

Tabla 4.3: Análisis de error porcentual en la banda de paso por efecto de rizado

Armónico	Err. Rect [%]	Err. Hamm [%]	Err. Kais [%]
H1	2.01	0.01	0.79
H2	0.36	0.05	1.20
H3	1.31	0.08	0.15
H4	1.36	0.07	0.24
H5	0.08	0.04	1.40
H6	2.74	0.02	0.69
H7	1.38	0.05	0.49
H8	4.20	0.02	1.15
H9	2.67	0.04	2.65
H10	3.66	5.22	0.13
Error Máximo	4.20	5.22	2.65
Error Medio	1.98	0.56	0.89

de este modo se optó por emplear la ventana de Kaiser que muestra un comportamiento más estable en ambas características que las otras dos opciones.

4.1.3 FILTRO PASABAJOS | RECUPERACIÓN DE FUNDAMENTAL

En última instancia se debe de diseñar un filtro pasa-bajos con tal de recuperar el armónico fundamental. Aquí se dispone de la misma problemática que en el caso anterior; sin embargo se dispone de la ventaja de que idóneamente se dispondrá de un único armónico, por lo que es posible compensar la ganancia del fenómeno de Gibbs que provoca la ventana rectangular. No obstante, se ha de evitar tener cambios muy abruptos ante posibles variaciones en la frecuencia de la red. De este modo se optó por emplear la misma ventana de Kaiser, ajustando el número de taps a 281, y un β de 2. Obteniendo así la siguiente respuesta en frecuencia:

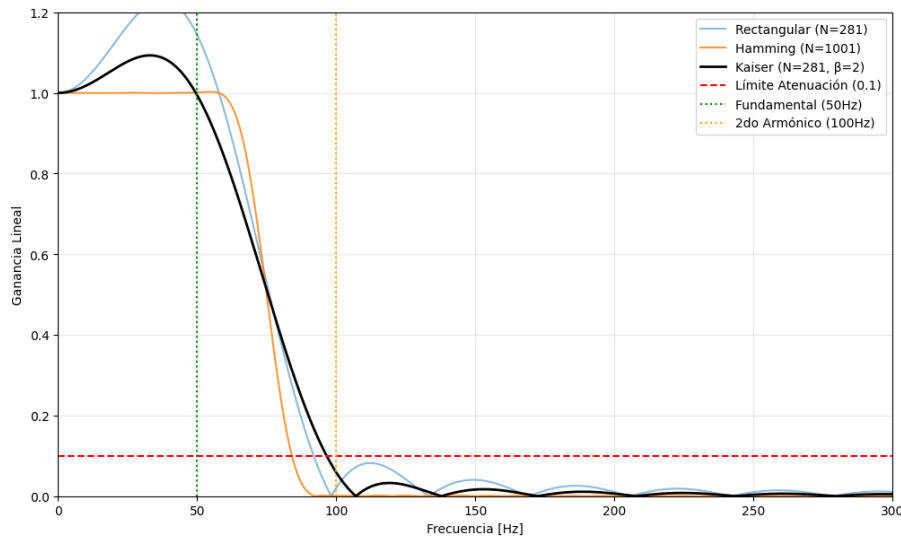


Figura 4.4: Módulo de respuesta en frecuencia de filtro pasa-bajos / Extracción de fundamental.

4.1.4 SIMULACIÓN

En última instancia se pide implementar el esquema completo y simularlo con la siguiente señal:

$$x[n] = A_1 \cdot \sin(2\pi \cdot n \cdot f_0 / f_s) + \sum_{k=2}^K A_k \cdot \sin(2\pi k f_0 \cdot n f_s) \quad (5)$$

donde $f_0 = 50Hz$, $f_s = 10kHz$ y A_k es el k -ésimo número de la sucesión de Fibonacci.

El THD de esta señal se puede calcular como:

$$THD = \frac{\sum_{k=2}^K 1/2 \cdot A_k^2}{1/2 \cdot A_1^2} \quad (6)$$

por lo que podemos emplear esta misma para validar el sistema diseñado hasta aquí. Para ello se realizó una simulación, donde se generó un segundo de esta señal y se la inyectó al sistema. Una vez que se acabó el transitorio se midieron los valores RMS de la señal de salida y se calculó el THD como:

$$THD_{medido} = \frac{\sqrt{V_t^2 - V_0^2}}{V_0} \quad (7)$$

Tabla 4.4: Resultados finales de la simulación del analizador de THD

Parámetro	Valor obtenido
Tensión RMS total (V_t - 10 armónicos)	0.8366
Tensión RMS fundamental (V_0)	0.7026
THD calculado por el algoritmo	64.64 %
THD teórico (Armónicos H_2 a H_{10})	65.29 %
Error relativo porcentual	0.99 %

donde el error observado esta asociado a las oscilaciones en la banda pasante de los filtros diseñados.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy import signal
4
5 # --- 1. Parametros Globales ---
6 fs = 10000
7 duration = 1.0 # 1 segundo
8 t = np.arange(0, duration, 1/fs)
9 f0 = 50
10
11 # --- 2. Generacion de Senal Sintetica (Fibonacci) ---
12 def fibonacci_amplitudes(n_harmonics):
13     fib = [1, 1]
14     for i in range(2, n_harmonics + 2):
15         fib.append(fib[-1] + fib[-2])
16     # Ak = 1 / f_{k+1}
17     return [1/fib[k] for k in range(1, n_harmonics + 1)]
18
19 # Generamos armonicos hasta Nyquist (fs/2 = 5000Hz -> 100 armonicos)
20 K = 100
21 A = fibonacci_amplitudes(K)
22 x = np.zeros_like(t)
23
24 for k in range(1, K + 1):
25     x += A[k-1] * np.sin(2 * np.pi * k * f0 * t)
26
27 # --- 3. Filtro Comb (y1) ---
28 # y1[n] = alpha*x[n] + beta*y1[n-k]
29 alpha = 2/11
30 beta = 9/11
31 k_delay = 200
32
33 # Implementacion recursiva del Comb
34 # b = [alpha], a = [1, 0, 0, ..., -beta]
35 b_comb = [alpha]
36 a_comb = np.zeros(k_delay + 1)
37 a_comb[0] = 1
38 a_comb[k_delay] = -beta
39 y1 = signal.lfilter(b_comb, a_comb, x)
40
41 # --- 4. Filtros FIR Kaiser (y2 e y3) ---
42 # y2: Recupera 10 armonicos
43 n2 = 281
44 beta2 = 1.8
45 fc2 = 530
46 taps_y2 = signal.firwin(n2, fc2, fs=fs, window=('kaiser', beta2))
47 y2 = signal.lfilter(taps_y2, [1.0], y1)
48
49 # y3: Recupera fundamental
50 n3 = 281
51 beta3 = 2.0
52 fc3 = 75
53 taps_y3 = signal.firwin(n3, fc3, fs=fs, window=('kaiser', beta3))
54 y3 = signal.lfilter(taps_y3, [1.0], y1)
55
56 # --- 5. Calculo de THD ---

```

```

57 # Para evitar el transitorio inicial de los filtros, descartamos el primer
    tramo
58 offset = 5000
59 y2_steady = y2[offset:]
60 y3_steady = y3[offset:]
61
62 # Valores RMS
63 Vt = np.sqrt(np.mean(y2_steady**2))
64 V0 = np.sqrt(np.mean(y3_steady**2))
65
66 # THD = sqrt(Vt^2 - V0^2) / V0
67 thd_calculado = (np.sqrt(Vt**2 - V0**2) / V0) * 100
68
69 # --- 6. Validacion Teorica ---
70 # THD_teorico usando los primeros 10 armonicos (A1 a A10)
71 sum_sq_harmonics = np.sum(np.array(A[1:10])**2)
72 thd_teorico = (np.sqrt(sum_sq_harmonics) / A[0]) * 100
73
74 print(A)
75 print(f"--- Resultados del Analizador ---")
76 print(f"V_total (10 armonicos): {Vt:.4f}")
77 print(f"V_fundamental (V0): {V0:.4f}")
78 print(f"THD calculado: {thd_calculado:.2f}%")
79 print(f"THD teorico (H2-H10): {thd_teorico:.2f}%")
80
81 # --- 7. Visualizacion ---
82 plt.figure(figsize=(12, 5))
83 plt.plot(t[offset:offset+1000], y2[offset:offset+1000], label='y2 (10
    Armonicos)')
84 plt.plot(t[offset:offset+1000], y3[offset:offset+1000], label='y3 (
    Fundamental)', linewidth=2)
85 plt.legend()
86 plt.grid(True)
87 plt.show()

```

Listing 4: Código empleado en la implementación.

REFERENCIAS

- [1] T. Instruments, *TL082 Wide Bandwidth JFET-Input Operational Amplifier Datasheet*, Último acceso: marzo de 2025, s.f. dirección: <https://www.ti.com/lit/ds/symmlink/tl082.pdf>
- [2] R. D. Middlebrook, *Measurement of Loop Gain in Feedback Systems*, International Journal of Electronics, Vol. 38, No. 4, pp. 485–512. Último acceso: marzo de 2025, 1975. dirección: https://www.venableinstruments.com/hubfs/Middlebrook_Measurement%20of%20loop%20gain%20in%20feedback%20systems.pdf
- [3] R. D. Middlebrook, *The GFT: A General Yet Practical Feedback Theorem*, Circuits and Systems Exposition (Review Version). Último acceso: marzo de 2026, 2002. dirección: <http://riddlebrook.com/wp-content/uploads/2014/05/The-GFT.pdf>
- [4] M. Tian, V. Visvanathan, J. Hantgan y K. Kundert, «Striving for Small-Signal Stability: Loop-Based and Device-Based Algorithms for Stability Analysis of Linear Analog Circuits in the Frequency Domain,» *IEEE Circuits and Devices Magazine*, vol. 17, n.º 1, págs. 31-41, ene. de 2001, Último acceso: marzo de 2025. DOI: 10.1109/101.900125 dirección: <https://ieeexplore.ieee.org/document/900125/>